Week 6 Wed Web Weather Monitor

Lab Date: Feb 18
Dude Date: 2:00pm Feb 23

Hardware Components: Arduino Uno, Raspberry Pi, breadboard, wires, MPU 6050, BH1750, DHT11,

Wiring Connections: Connected 5V power supply to motor through the breadboard, ground is connected to the breadboard which is connected to the arduino. MPU 6050, BH1750 both connected through sda and scl to the arduino. DHT11 is connected via digital pin 2.

Write the following code to the arduino:

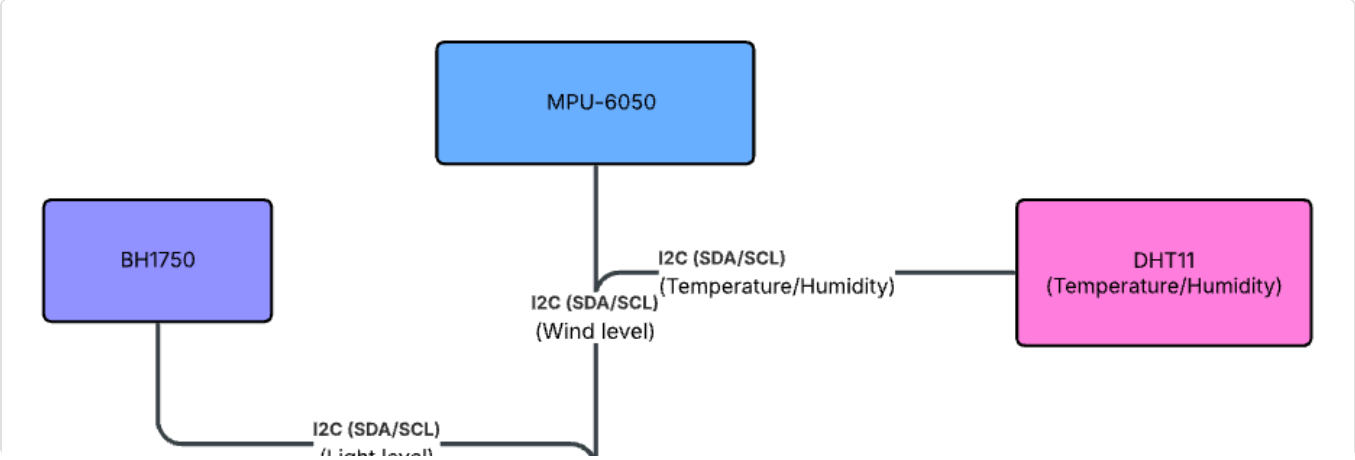
```
//DHT Lib

#include <TinyDHT.h>



//MPU Libs

#include <Adafruit_MPU6050.h>

#include <Adafruit_Sensor.h>

#include <Wire.h>



//BH1750 Libs

#include <BH1750.h>




//DHT Def

#define DHT11_PIN 2
```

```
DHT dht11(DHT11_PIN, DHT11);



//MPU Defs

Adafruit_MPU6050 mpu;

float windLevel = 0;

float factor = 10;



//BH1750 Defs

BH1750 lightMeter;



void setup() {

  Serial.begin(115200);



  //CODE FOR DHT_____________________________________

  dht11.begin();



  //CODE FOR MPU _____________________________________

  if (!mpu.begin()) {

    Serial.println("Failed to find MPU6050 chip");

    while (1) {

      delay(10);

    }

  }
```

```
  Serial.println("MPU6050 Found!");


  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);




  //CODE FOR BH1750_______________________________
  Wire.begin();
  lightMeter.begin();
  Serial.println("BH1750 Test Begin");
  delay(100);


}


void loop() {


  //CODE FOR DTH ___________________________
  //wait a few seconds between measurements
  delay(2000);


  float humi = dht11.readHumidity();
  float tempC = dht11.readTemperature();
```

```
if(isnan(humi) || isnan(tempC)){
  Serial.println("Failed to read from DHT11 sensor");
}
  Serial.print("DHT11# Humidity: ");
  Serial.print(humi);
  Serial.print("%");


  Serial.print("  |  ");


  Serial.print("Temperature: ");
  Serial.print(tempC);
  Serial.print("°C ~ ");
  Serial.print("\n");


  //CODE FOR MPU ________________________________
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);
  float ax = a.acceleration.x;
  float ay = a.acceleration.y;
  float az = a.acceleration.z;
  windLevel = factor*ax + factor*ay + factor*az;
```

```
    Serial.print("Wind Level: ");

    Serial.println(windLevel);



    //CODE FOR BH1750

    float lux = lightMeter.readLightLevel();

    Serial.print("Light: ");

    Serial.print(lux);

    Serial.println(" lx");

    delay(1000);

}
```



1. Receive Sensor Data

The following the is HTML and Flask code for the weather monitor website. It refreshes every five seconds and it takes the data from the sensors and plots them. The data is loaded from the arduino in JSON format which is then converted into a CSV file. The CSV file is loaded into the graph and the graph is refreshed every 5 seconds.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Web Weather Monitor</title>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.1/chart.umd.min.js"
></script>
<link href="https://fonts.googleapis.com/css2?
family=DM+Sans:wght@300;400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
<style>
  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

  :root {
    --bg: #f4f6f9;
    --surface: #ffffff;
    --border: #e2e8f0;
    --text-primary: #1a202c;
    --text-secondary: #64748b;
    --text-muted: #94a3b8;
    --accent: #3b82f6;
    --accent-light: rgba(59,130,246,0.08);
    --accent-fill: rgba(59,130,246,0.12);
    --green: #22c55e;
    --shadow: 0 1px 3px rgba(0,0,0,0.06), 0 1px 2px rgba(0,0,0,0.04);
```

```
    --shadow-lg: 0 4px 16px rgba(0,0,0,0.08);
    --radius: 12px;
  }

  body {
    font-family: 'DM Sans', sans-serif;
    background: var(--bg);
    color: var(--text-primary);
    min-height: 100vh;
    padding: 24px;
  }

  /* ── Header ── */
  .header {
    display: flex;
    align-items: flex-start;
    justify-content: space-between;
    margin-bottom: 24px;
  }
  .header h1 { font-size: 1.5rem; font-weight: 700; letter-spacing: -0.02em;
}
  .header .subtitle { font-size: 0.8rem; color: var(--text-secondary);
margin-top: 2px; }

  .status-badge {
    display: flex; align-items: center; gap: 6px;
    font-size: 0.78rem; color: var(--text-secondary);
    background: var(--surface); border: 1px solid var(--border);
    border-radius: 20px; padding: 5px 12px;
    box-shadow: var(--shadow);
  }
  .status-dot {
    width: 8px; height: 8px; border-radius: 50%;
    background: var(--text-muted);
    transition: background 0.3s;
  }
  .status-dot.loaded { background: var(--green); }

  /* ── Stat Cards ── */
  .stat-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 12px;
    margin-bottom: 20px;
  }
  .stat-card {
```

```
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    padding: 18px 20px;
    box-shadow: var(--shadow);
    transition: box-shadow 0.2s;
  }
  .stat-card:hover { box-shadow: var(--shadow-lg); }
  .stat-label {
    font-size: 0.68rem; letter-spacing: 0.08em;
    text-transform: uppercase; color: var(--text-muted);
    font-weight: 600; margin-bottom: 8px;
  }
  .stat-main {
    display: flex; align-items: baseline;
    justify-content: space-between; gap: 8px;
    margin-bottom: 6px;
  }
  .stat-value { font-size: 1.7rem; font-weight: 700; letter-spacing:
-0.03em; }
  .stat-unit { font-size: 0.85rem; color: var(--text-secondary); font-
weight: 400; margin-left: 2px; }
  .stat-delta {
    font-size: 0.82rem; font-weight: 600;
    color: var(--accent); font-family: 'DM Mono', monospace;
  }
  .stat-range { font-size: 0.75rem; color: var(--text-muted); }
  .stat-range span { color: var(--text-secondary); }

  /* ── Upload Bar ── */
  .upload-bar {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    padding: 14px 20px;
    margin-bottom: 20px;
    display: flex; align-items: center; gap: 14px; flex-wrap: wrap;
    box-shadow: var(--shadow);
  }
  .upload-bar label {
    font-size: 0.82rem; font-weight: 600;
    color: var(--text-secondary); white-space: nowrap;
  }
  .upload-btn {
    display: inline-flex; align-items: center; gap: 6px;
    background: var(--accent); color: white;
```

```
    border: none; border-radius: 8px; padding: 8px 16px;
    font-family: 'DM Sans', sans-serif; font-size: 0.82rem; font-weight:
600;
    cursor: pointer; transition: all 0.2s;
  }
  .upload-btn:hover { background: #2563eb; transform: translateY(-1px); box-
shadow: 0 4px 12px rgba(59,130,246,0.35); }
  #csv-input { display: none; }
  .csv-hint {
    font-size: 0.75rem; color: var(--text-muted);
    font-family: 'DM Mono', monospace;
  }

  /* ── Chart Panel ── */
  .chart-panel {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    padding: 24px;
    box-shadow: var(--shadow);
  }
  .chart-header {
    display: flex; align-items: center;
    justify-content: space-between; margin-bottom: 20px;
  }
  .chart-title { font-size: 0.95rem; font-weight: 700; }
  .metric-select {
    border: 1px solid var(--border); border-radius: 8px;
    padding: 7px 32px 7px 12px;
    font-family: 'DM Sans', sans-serif; font-size: 0.82rem;
    color: var(--text-primary); background: var(--bg);
    cursor: pointer; appearance: none;
    background-image: url("data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' width='12' height='12' viewBox='0 0 24
24' fill='none' stroke='%2364748b' stroke-width='2'%3E%3Cpath d='M6 9l6 6 6-
6'/%3E%3C/svg%3E");
    background-repeat: no-repeat; background-position: right 10px center;
  }
  .chart-wrap { position: relative; height: 340px; }

  .note { font-size: 0.72rem; color: var(--text-muted); margin-top: 14px;
font-style: italic; }

  .tags { margin-top: 12px; display: flex; gap: 8px; flex-wrap: wrap; }
  .tag {
    font-size: 0.7rem; font-weight: 600; letter-spacing: 0.04em;
```

```
    background: var(--bg); border: 1px solid var(--border);
    border-radius: 6px; padding: 3px 10px;
    color: var(--text-secondary);
  }

  /* empty state */
  .empty-state {
    position: absolute; inset: 0;
    display: flex; flex-direction: column; align-items: center; justify-
content: center;
    color: var(--text-muted); gap: 8px; font-size: 0.85rem;
  }
  .empty-state svg { opacity: 0.3; }
</style>
</head>
<body>

<div class="header">
  <div>
    <h1>Web Weather Monitor</h1>
    <div class="subtitle">Upload a CSV to visualise your logged data</div>
  </div>
  <div class="status-badge">
    <div class="status-dot" id="status-dot"></div>
    <span id="status-text">No data loaded</span>
  </div>
</div>

<!-- Stat Cards -->
<div class="stat-grid">
  <div class="stat-card">
    <div class="stat-label">Latest Timestamp</div>
    <div class="stat-value" id="ts-val" style="font-size:1.35rem">—</div>
    <div class="stat-range" id="ts-source">Source: —</div>
  </div>
  <div class="stat-card">
    <div class="stat-label">Temperature</div>
    <div class="stat-main">
      <div><span class="stat-value" id="temp-val">—</span><span class="stat-
unit">°C</span></div>
      <div class="stat-delta" id="temp-delta">—</div>
    </div>
    <div class="stat-range">Last hour <span id="temp-range">—</span></div>
  </div>
  <div class="stat-card">
    <div class="stat-label">Humidity</div>
```

```
    <div class="stat-main">
      <div><span class="stat-value" id="hum-val">—</span><span class="stat-
unit">%</span></div>
      <div class="stat-delta" id="hum-delta">—</div>
    </div>
    <div class="stat-range">Last hour <span id="hum-range">—</span></div>
  </div>
  <div class="stat-card">
    <div class="stat-label">Wind Level</div>
    <div class="stat-main">
      <div><span class="stat-value" id="wind-val">—</span><span class="stat-
unit">arb.</span></div>
      <div class="stat-delta" id="wind-delta">—</div>
    </div>
    <div class="stat-range">Last hour <span id="wind-range">—</span></div>
  </div>
</div>

<!-- Chart -->
<div class="chart-panel">
  <div class="chart-header">
    <div class="chart-title">Value vs. Time</div>
    <select class="metric-select" id="metric-select"
onchange="renderChart()"></select>
  </div>
  <div class="chart-wrap">
    <div class="empty-state" id="empty-state">
      <svg width="48" height="48" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.2"><polyline points="22 12 18 12 15 21
9 3 6 12 2 12"/></svg>
      Upload a CSV file to see the chart
    </div>
    <canvas id="myChart" style="display:none"></canvas>
  </div>
  <div class="note" id="chart-note" style="display:none">
    Tip: the timestamp column should be parseable date/time strings or Unix
timestamps.
  </div>
  <div class="tags">
    <span class="tag">HTML + CSS</span>
    <span class="tag">Chart.js</span>
    <span class="tag">CSV Upload</span>
  </div>
</div>

<script>
```

```
let parsedData = null; // { labels: [], columns: { colName: [values] } }
let chart = null;

function handleCSV(input) {
  const file = input.files[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = e => {
    try {
      parseCSV(e.target.result, file.name);
    } catch(err) {
      alert('Error parsing CSV: ' + err.message);
    }
  };
  reader.readAsText(file);
}

function parseCSV(text, filename) {
  const lines = text.trim().split(/\r?\n/).filter(l => l.trim());
  if (lines.length < 2) throw new Error('CSV must have at least a header row
and one data row.');

  // Auto-detect delimiter
  const firstLine = lines[0];
  const delim = firstLine.includes('\t') ? '\t' : ',';

  const headers = firstLine.split(delim).map(h =>
h.trim().replace(/^"|"$/g,''));

  const rows = lines.slice(1).map(l => l.split(delim).map(v =>
v.trim().replace(/^"|"$/g,'')));

  // Find timestamp column (first column that looks like dates or is named
timestamp/time/date)
  const tsKeywords = ['timestamp','time','date','datetime','ts'];
  let tsIdx = headers.findIndex(h => tsKeywords.some(k =>
h.toLowerCase().includes(k)));
  if (tsIdx === -1) tsIdx = 0; // fallback: first column

  // Build labels
  const labels = rows.map(r => {
    const raw = r[tsIdx];
    if (!raw) return '';
    const d = new Date(isNaN(raw) ? raw : Number(raw) * (raw.length <= 10 ?
1000 : 1));
    if (!isNaN(d)) {
```

```
      return d.toLocaleTimeString([], {hour:'2-digit', minute:'2-digit'});
    }
    return raw;
  });

  // Numeric columns (excluding timestamp)
  const columns = {};
  headers.forEach((h, i) => {
    if (i === tsIdx) return;
    const vals = rows.map(r => parseFloat(r[i]));
    if (vals.some(v => !isNaN(v))) {
      columns[h] = vals;
    }
  });

  if (Object.keys(columns).length === 0) throw new Error('No numeric columns
found in CSV.');

  parsedData = { labels, columns, filename };

  // Populate metric selector
  const sel = document.getElementById('metric-select');
  sel.innerHTML = '';
  Object.keys(columns).forEach(k => {
    const opt = document.createElement('option');
    opt.value = k; opt.textContent = k;
    sel.appendChild(opt);
  });

  // Map known columns to stats cards
  updateStatCards(columns, labels, filename);

  document.getElementById('status-dot').classList.add('loaded');
  document.getElementById('status-text').textContent = 'Data loaded';

  renderChart();
}

function updateStatCards(columns, labels, filename) {
  // Latest timestamp
  document.getElementById('ts-val').textContent = labels[labels.length-1] ||
'—';
  document.getElementById('ts-source').textContent = 'Source: ' + filename;

  function updateCard(keywords, valId, deltaId, rangeId, decimals=1) {
    const key = Object.keys(columns).find(k => keywords.some(kw =>
```

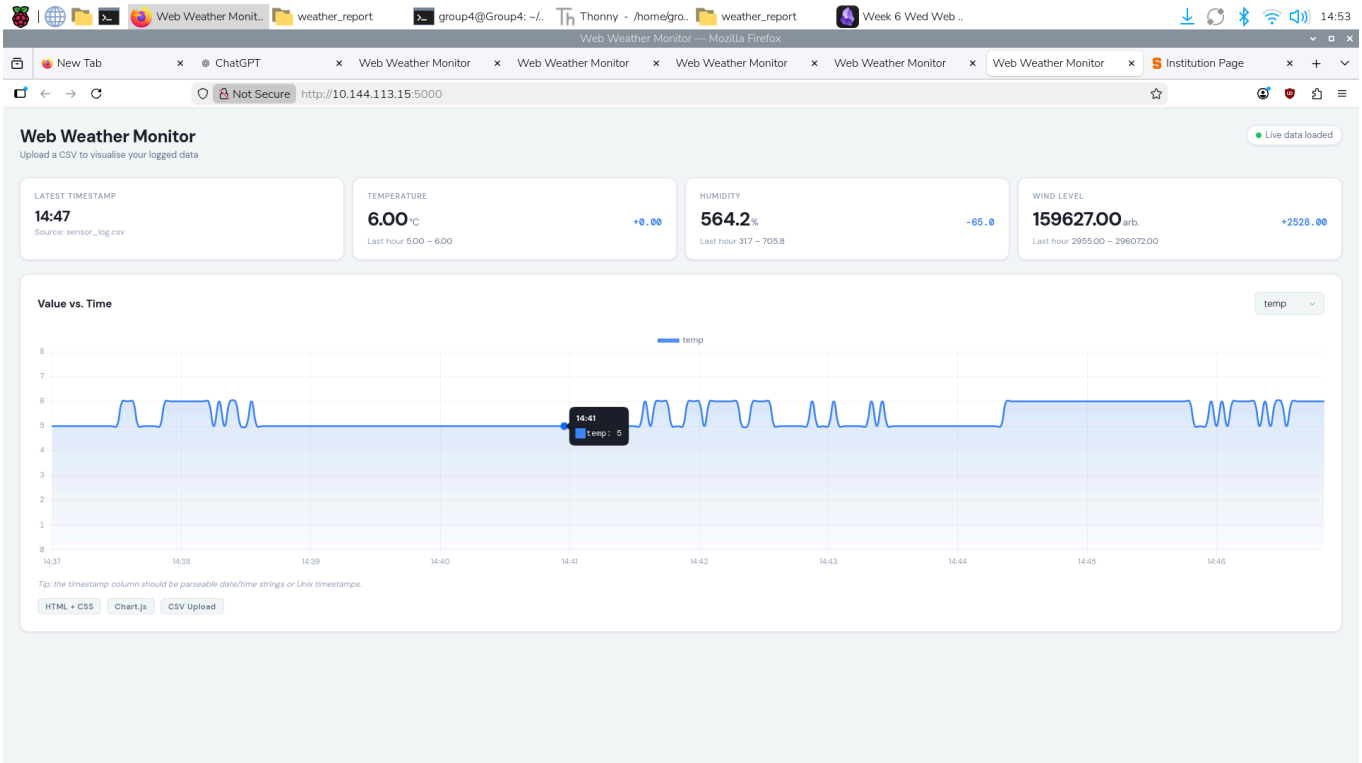
```
k.toLowerCase().includes(kw)));
    if (!key) return;
    const vals = columns[key].filter(v => !isNaN(v));
    if (!vals.length) return;
    const last = vals[vals.length-1];
    const prev = vals.length > 1 ? vals[vals.length-2] : last;
    const delta = last - prev;
    const min = Math.min( ... vals).toFixed(decimals);
    const max = Math.max( ... vals).toFixed(decimals);
    document.getElementById(valId).textContent = last.toFixed(decimals);
    document.getElementById(deltaId).textContent = (delta >= 0 ? '+' : '') +
delta.toFixed(decimals);
    document.getElementById(rangeId).textContent = min + ' — ' + max;
  }

  updateCard(['temp'],'temp-val','temp-delta','temp-range',2);
  updateCard(['hum'],'hum-val','hum-delta','hum-range',1);
  updateCard(['wind'],'wind-val','wind-delta','wind-range',2);
}

function renderChart() {
  if (!parsedData) return;
  const sel = document.getElementById('metric-select');
  const metric = sel.value;
  if (!metric || !parsedData.columns[metric]) return;

  const vals = parsedData.columns[metric];
  const labels = parsedData.labels;

  document.getElementById('empty-state').style.display = 'none';
  document.getElementById('chart-note').style.display = 'block';
  const canvas = document.getElementById('myChart');
  canvas.style.display = 'block';

  if (chart) chart.destroy();

  chart = new Chart(canvas, {
    type: 'line',
    data: {
      labels,
      datasets: [{
        label: metric,
        data: vals,
        borderColor: '#3b82f6',
        backgroundColor: ctx => {
          const gradient = ctx.chart.ctx.createLinearGradient(0, 0, 0,
```

```
ctx.chart.height);
          gradient.addColorStop(0, 'rgba(59,130,246,0.25)');
          gradient.addColorStop(1, 'rgba(59,130,246,0.0)');
          return gradient;
        },
        borderWidth: 2.5,
        pointRadius: vals.length > 100 ? 0 : 3,
        pointHoverRadius: 5,
        pointBackgroundColor: '#3b82f6',
        fill: true,
        tension: 0.4,
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      interaction: { mode: 'index', intersect: false },
      plugins: {
        legend: {
          labels: {
            font: { family: 'DM Sans', size: 12 },
            color: '#64748b',
            boxWidth: 28, boxHeight: 3,
          }
        },
        tooltip: {
          backgroundColor: '#1a202c',
          titleFont: { family: 'DM Sans', size: 12 },
          bodyFont: { family: 'DM Mono', size: 12 },
          padding: 10,
          cornerRadius: 8,
        }
      },
      scales: {
        x: {
          grid: { color: 'rgba(0,0,0,0.04)' },
          ticks: {
            font: { family: 'DM Sans', size: 11 },
            color: '#94a3b8',
            maxTicksLimit: 10,
          },
          border: { display: false }
        },
        y: {
            min:0,
            max: 8,
```

```
          grid: { color: 'rgba(0,0,0,0.04)' },
          ticks: {
            font: { family: 'DM Mono', size: 11 },
            color: '#94a3b8',
          },
          border: { display: false }
        }
      }
    }
  });
}
async function loadFromServer() {
  try {
    const res = await fetch('/history');
    const data = await res.json();

    if (!data.length) return;

    const labels = data.map(row => {
      const d = new Date(row.timestamp);
      return d.toLocaleTimeString([], {hour:'2-digit', minute:'2-digit'});
    });

    const columns = {};

    Object.keys(data[0]).forEach(key => {
      if (key === "timestamp") return;
      columns[key] = data.map(row => parseFloat(row[key]));
    });

    parsedData = { labels, columns };

    // Populate dropdown
    const sel = document.getElementById('metric-select');
    sel.innerHTML = '';
    Object.keys(columns).forEach(k => {
      const opt = document.createElement('option');
      opt.value = k;
      opt.textContent = k;
      sel.appendChild(opt);
    });

    updateStatCards(columns, labels, "sensor_log.csv");

    document.getElementById('status-dot').classList.add('loaded');
    document.getElementById('status-text').textContent = 'Live data loaded';
```

```
    renderChart();

  } catch (err) {
    console.error("Error loading data:", err);
  }
}
window.onload = function() {
  loadFromServer();

  // Optional: refresh every 5 seconds
  setInterval(loadFromServer, 5000);
};
</script>
</body>
</html>
```

```
from flask import Flask, render_template
import serial
import json
import pandas as pd
import os
import time
from datetime import datetime
import threading

app = Flask(__name__)

PORT = '/dev/ttyACM1'   # Change if needed (e.g. COM3 on Windows)
BAUD = 115200
CSV_FILE = "sensor_log.csv"

# Create CSV if it doesn't exist
if not os.path.exists(CSV_FILE):
    sensor_data["timestamp"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    ordered_row = [
        sensor_data.get("temp"),
        sensor_data.get("humidity"),
        sensor_data.get("lux"),
        sensor_data.get("wind"),
        sensor_data.get("timestamp")
    ]

    df = pd.DataFrame([ordered_row], columns=["temp", "humidity", "lux",
```

```
"wind", "timestamp"])
    df.to_csv(CSV_FILE, mode='a', header=False, index=False)

def serial_logger():
    try:
        ser = serial.Serial(PORT, BAUD, timeout=1)
        time.sleep(2)  # Allow Arduino to reset
        print("Serial connection established")

        while True:
            try:
                line = ser.readline().decode().strip()

                if not line:
                    continue

                print("Raw:", line)

                sensor_data = json.loads(line)
                sensor_data["timestamp"] = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")

                df = pd.DataFrame([sensor_data])
                df.to_csv(CSV_FILE, mode='a', header=False, index=False)

                print("Logged:", sensor_data)

            except json.JSONDecodeError:
                print("Invalid JSON received")
            except Exception as e:
                print("Logging error:", e)

    except Exception as e:
        print("Serial connection failed:", e)

@app.route('/')
def index():
    return render_template("index.html")

@app.route('/history')
def history():
    try:
        df = pd.read_csv(CSV_FILE)
        return df.to_json(orient="records")
    except Exception as e:
        return {"error": str(e)}
```

```python
if __name__ == '__main__':
    # Start background serial logging thread
    thread = threading.Thread(target=serial_logger)
    thread.daemon = True
    thread.start()

    app.run(host='0.0.0.0', port=5000)
```